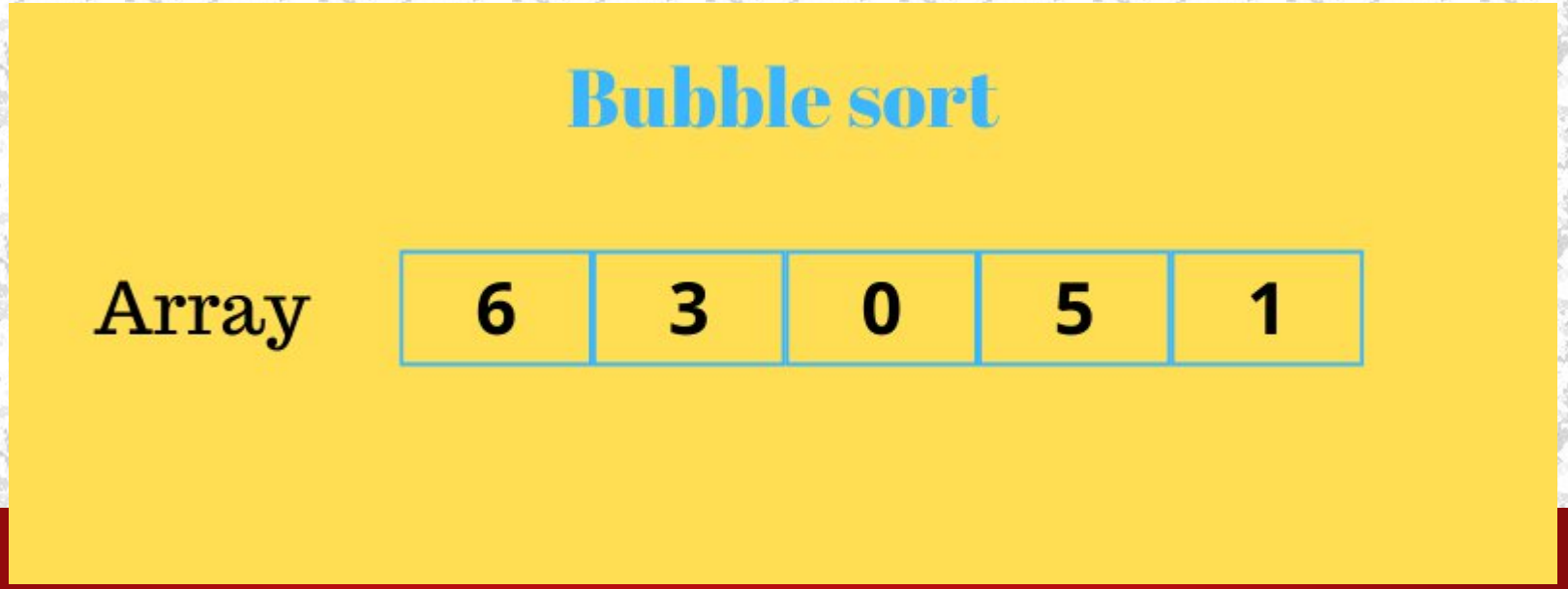
BUBBLE AND INSERTION SORTING : LECTURE

DSAA 211: WEEK 8



BUBBLE SORTING

- A SORTING ALGORITHM THAT COMPARES TWO ADJACENT ELEMENTS AND SWAPS THEM UNTIL THEY ARE IN THE INTENDED ORDER.



HOW BUBBLE SORT WORKS?



- *Start at the first element of the array.*
- *Compare the current element with the next element.*
- *If the current element is greater than the next element, swap them.*
- *Move to the next pair of elements and repeat the comparison and swap if needed.*
- *After each complete pass through the array, the largest unsorted element is placed at its correct position at the end of the array.*
- *Repeat the above process of a pass for the remaining unsorted elements until the entire array is sorted.*

EXAMPLE:

//29, 10, 14, 37, 13

Pass	Comparisons & Swaps	Array State
1	Compare 29 & 10 → swap	{10, 29, 14, 37, 13}
	Compare 29 & 14 → swap	{10, 14, 29, 37, 13}
	Compare 29 & 37 → no swap	{10, 14, 29, 37, 13}
	Compare 37 & 13 → swap	{10, 14, 29, 13, 37}
2	Compare 10 & 14 → no swap	{10, 14, 29, 13, 37}
	Compare 14 & 29 → no swap	{10, 14, 29, 13, 37}
	Compare 29 & 13 → swap	{10, 14, 13, 29, 37}
3	Compare 10 & 14 → no swap	{10, 14, 13, 29, 37}
	Compare 14 & 13 → swap	{10, 13, 14, 29, 37}
4	Compare 10 & 13 → no swap	{10, 13, 14, 29, 37}

TIME COMPLEXITY

Type	Complexity	Description
Best Case	O(n)	Occurs when the list is already sorted; only one pass is needed (with an optimized version that detects no swaps).
Average / Worst Case	$O(n^2)$	Happens when the list is unsorted or reversed; requires multiple passes and comparisons for each element.
Space Complexity	O(1)	Sorting is done in place using only a few temporary variables, so no extra memory is required.

LET'S PLAY : SORT IT RIGHT!

Work together to sort a list of unsorted numbers in ascending order, using one swap at a time.

They must count:

- ✓ Number of swaps (how many times you exchange two numbers)
- ✓ Number of passes (how many rounds it took to complete sorting)

GAME MECHANICS

1. Sort one pair at a time — just like in Bubble Sort.
2. You must compare two adjacent numbers, and if they are in the wrong order, swap them.
3. Continue the process until all numbers are in ascending order.
4. Once sorted, Raise your hands or call "Sorted!"

SCORING

Criteria	Description	Points
Correctly sorted list	All numbers are in proper ascending order	20 pts
Accurate swap count	Group correctly counted how many swaps occurred	10 pts
Accurate pass count	Correct number of passes identified	10 pts
Teamwork & participation	All group members contributed	5 pts
Fastest		5 pts
Total		50 pts

PRACTICE

Unsorted List:

```
[2, 4, 1, 3]
```

Pass	Comparison	Swap?	List After Pass	Swaps So Far
1	2 > 4	✗	[2, 4, 1, 3]	0
1	4 > 1	✓	[2, 1, 4, 3]	1
1	4 > 3	✓	[2, 1, 3, 4]	2
2	2 > 1	✓	[1, 2, 3, 4]	3

✓ Total Swaps: 3

✓ Total Pass: 2

GAME 1

Unsorted List:

`[8, 3, 6, 2, 5]`

Pass	Comparison	Swap?	List After Pass	Swaps This Pass
1	8 > 3	✓	[3, 8, 6, 2, 5]	1
1	8 > 6	✓	[3, 6, 8, 2, 5]	2
1	8 > 2	✓	[3, 6, 2, 8, 5]	3
1	8 > 5	✓	[3, 6, 2, 5, 8]	4
2	3 > 6	×	[3, 6, 2, 5, 8]	0
2	6 > 2	✓	[3, 2, 6, 5, 8]	5
2	6 > 5	✓	[3, 2, 5, 6, 8]	6
3	3 > 2	✓	[2, 3, 5, 6, 8]	7

✓ Total Swaps: 7

✓ Total Pass: 3

GAME 2

Unsorted List:

```
[9, 7, 4, 3, 1]
```

Pass	Comparison	Swap?	List After Pass	Swaps So Far
1	9 > 7	✔	[7, 9, 4, 3, 1]	1
1	9 > 4	✔	[7, 4, 9, 3, 1]	2
1	9 > 3	✔	[7, 4, 3, 9, 1]	3
1	9 > 1	✔	[7, 4, 3, 1, 9]	4
2	7 > 4	✔	[4, 7, 3, 1, 9]	5
2	7 > 3	✔	[4, 3, 7, 1, 9]	6
2	7 > 1	✔	[4, 3, 1, 7, 9]	7
3	4 > 3	✔	[3, 4, 1, 7, 9]	8
3	4 > 1	✔	[3, 1, 4, 7, 9]	9
4	3 > 1	✔	[1, 3, 4, 7, 9]	10

✔ Total Swaps: 10

✔ Total Pass: 4

GAME 3

Unsorted List:

[12, 6, 9, 3, 15, 8, 1, 5]

Pass	List After Each Pass	Swaps This Pass
1	[6, 9, 3, 12, 8, 1, 5, 15]	5
2	[6, 3, 9, 8, 1, 5, 12, 15]	5
3	[3, 6, 8, 1, 5, 9, 12, 15]	3
4	[3, 6, 1, 5, 8, 9, 12, 15]	2
5	[3, 1, 5, 6, 8, 9, 12, 15]	1
6	[1, 3, 5, 6, 8, 9, 12, 15]	1
✔ Total Swaps: 17		
✔ Total Passes: 6		

GENERAL ALGORITHM

Step 1: For i = 0 to N-1 repeat Step 2
Step 2: For J = i + 1 to N – I repeat
Step 3: if A[J] > A[i]
Swap A[J] and A[i]
[End of Inner for loop]
[End if Outer for loop]
Step 4: Exit

CONCLUSION

- SORTING IS ONE OF THE MOST USED TECHNIQUES BY PROGRAMMERS TO STORE AND OPTIMIZE A HUGE AMOUNT OF DATA.
- BY SORTING THE ELEMENTS, YOU CAN PROVIDE A BETTER VISUAL TO YOUR DATA COLLECTION AND REDUCE THE TIME COMPLEXITY OF ANY PROGRAM EFFICIENTLY.
- BUBBLE SORT IS NOT COMMONLY USED COMPARED TO OTHER SORTING ALGORITHMS BUT STILL HAVE ITS OWN IMPORTANCE IN THE TECHNICAL WORLD. LEARNING BUBBLE SORT IS WELL WORTH THE EFFORT BECAUSE IT ACTS AS A BRIDGE TO UNDERSTANDING HIGHER-LEVEL CONCEPTS AND MAKES IT EASIER FOR YOU TO UNDERSTAND MORE DIFFICULT TOPICS.

INSERTION SORTING

- INSERTION SORT IS A SIMPLE SORTING ALGORITHM THAT WORKS BY ITERATIVELY INSERTING EACH ELEMENT OF AN UNSORTED LIST INTO ITS CORRECT POSITION IN A SORTED PORTION OF THE LIST.

It is a stable sorting algorithm, meaning that elements with equal values maintain their relative order in the sorted output.



HOW INSERTION SORT WORKS?

- We start with second element of the array as first element in the array is assumed to be sorted.
- Compare second element with the first element and check if the second element is smaller then swap them.
- Move to the third element and compare it with the second element, then the first element and swap as necessary to put it in the correct position among the first three elements.
- Continue this process, comparing each element with the ones before it and swapping as needed to place it in the correct position among the sorted elements.
- Repeat until the entire array is sorted.



//29, 10, 14, 37, 13

EXAMPLE:

Pass (i)	Key Value	Comparison/Shift	Array State
Start	–	–	{29, 10, 14, 37, 13}
i = 1	10	Compare 29 > 10 → shift 29 right	{29, 29, 14, 37, 13}
		Insert 10 at position 0	{10, 29, 14, 37, 13}
i = 2	14	Compare 29 > 14 → shift 29 right	{10, 29, 29, 37, 13}
		Compare 10 < 14 → stop	{10, 14, 29, 37, 13}
i = 3	37	Compare 29 < 37 → stop	{10, 14, 29, 37, 13}
i = 4	13	Compare 37 > 13 → shift 37 right	{10, 14, 29, 37, 37}
		Compare 29 > 13 → shift 29 right	{10, 14, 29, 29, 37}
		Compare 14 > 13 → shift 14 right	{10, 14, 14, 29, 37}
		Compare 10 < 13 → stop, insert	{10, 13, 14, 29, 37}

TIME COMPLEXITY

Type	Complexity	Description
Best Case	O(n)	Happens when the list is already sorted. Only one comparison per element.
Average / Worst Case	$O(n^2)$	Occurs when elements are in random or reverse order; uses nested loops to compare and shift.
Space Complexity	O(1)	Performs sorting in place, requiring no extra memory beyond the given array.

LET'S PLAY : INSERT IT RIGHT!

Teams will race to correctly sort a set of unsorted numbers using the Insertion Sort process — one number at a time — while following the correct algorithmic steps.

GAME MECHANICS

1. Each team receives identical unsorted cards, for example: [8, 3, 6, 2, 7, 4]
2. When the teacher says “GO!”, teams start sorting using Insertion Sort rules:
   - The first card is automatically “sorted.”
   - Take the next card and insert it in the correct position within the sorted section.
   - Continue one card at a time until all are sorted.
3. Only one member may move cards at a time — the others help by deciding where to insert.
4. Once a team finishes, they must raise their hand and shout “SORTED!”

SCORING

Criteria	Points	Description
Accuracy	40	Final sorted list is 100% correct
Correct Algorithm Process	25	Followed Insertion Sort (inserted one card at a time)
Insertions #	15	Correct # of Insertions
Speed	10	Sorted quickly compared to other groups
Pass #	10	Correct # of Pass

PRACTICE

Starting Cards

`[5, 2, 4, 3, 1]`

Pass #	List After Pass	Action Taken	# of Swaps
1	`[2, 5, 4, 3, 1]`	Insert 2 before 5	1
2	`[2, 4, 5, 3, 1]`	Insert 4 between 2 and 5	1
3	`[2, 3, 4, 5, 1]`	Insert 3 between 2 and 4	2
4	`[1, 2, 3, 4, 5]`	Insert 1 at the beginning	4
✔ Final	`[1, 2, 3, 4, 5]`	Sorted!	8 Total Swaps

GAME 1: Unsorted List: [9, 5, 1, 4, 3, 8, 2]

Pass #	List After Pass	Action Taken	# of Swaps
1	[5, 9, 1, 4, 3, 8, 2]	Insert 5 before 9	1
2	[1, 5, 9, 4, 3, 8, 2]	Insert 1 at the start	2
3	[1, 4, 5, 9, 3, 8, 2]	Insert 4 between 1 and 5	1
4	[1, 3, 4, 5, 9, 8, 2]	Insert 3 between 1 and 4	2
5	[1, 3, 4, 5, 8, 9, 2]	Insert 8 before 9	1
6	[1, 2, 3, 4, 5, 8, 9]	Insert 2 between 1 and 3	5
✔ Final	[1, 2, 3, 4, 5, 8, 9]	Sorted!	12 Total Swaps

GAME 2: Unsorted List: [7, 3, 5, 9, 1, 8, 2, 6, 4]

Pass #	List After Pass	Action Taken	# of Swaps
1	[3, 7, 5, 9, 1, 8, 2, 6, 4]	Insert 3 before 7	1
2	[3, 5, 7, 9, 1, 8, 2, 6, 4]	Insert 5 between 3 and 7	1
3	[3, 5, 7, 9, 1, 8, 2, 6, 4]	9 already in correct place	0
4	[1, 3, 5, 7, 9, 8, 2, 6, 4]	Insert 1 at start	4
5	[1, 3, 5, 7, 8, 9, 2, 6, 4]	Insert 8 before 9	1
6	[1, 2, 3, 5, 7, 8, 9, 6, 4]	Insert 2 between 1 and 3	5
7	[1, 2, 3, 5, 6, 7, 8, 9, 4]	Insert 6 between 5 and 7	3
8	[1, 2, 3, 4, 5, 6, 7, 8, 9]	Insert 4 between 3 and 5	4
✔ Final	[1, 2, 3, 4, 5, 6, 7, 8, 9]	Sorted!	19 Total Swaps

GENERAL ALGORITHM

Step 1: Repeat Steps 2 to 5 for K = 1 to N-1

Step 2: set temp = A[K]

Step 3: set J = K – 1

Step 4: Repeat while temp <=A[J]

set A[J + 1] = A[J]

set J = J – 1

[end of inner loop]

Step 5: set A[J + 1] = temp

[end of loop]

Step 6: exit

WHEN TO USE INSERTION SORT

- ✓ When the dataset is small.
- ✓ When the data is almost sorted.
- ✓ When simplicity and readability matter more than speed.

BUBBLE AND INSERTION SORTING : LABORATORY

DSAA 211: WEEK 8



BUBBLE SORTING : LABORATORY

DSAA 211: WEEK 8



PRACTICE 1 : BUBBLE SORT

```
//initialization
    int arr[5] = {29, 10, 14, 37, 13};
    int n = 5;
    int temp;
    int swapCount = 0;
//display the arrays
    printf("Original array: ");
    for(int i = 0; i < n; i++) {
        printf("%d ", arr[i]);
    }
```

```c
//process
    for(int i = 0; i < n - 1; i++) {
        for(int j = 0; j < n - i - 1; j++) {
            if(arr[j] > arr[j + 1]) {
                //swap elements
                temp = arr[j];
                arr[j] = arr[j + 1];
                arr[j + 1] = temp;
                swapCount++;


                //print swap elements
                printf("Swap %d: ", swapCount);
                for(int k = 0; k < n; k++) {
                    printf("%d ", arr[k]);
                }
                printf("\n");
            }
        }
    }
```

```c
//result
    printf("\nSorted array: ");
    for(int i = 0; i < n; i++) {
        printf("%d ", arr[i]);
    }
    //total
    printf("\nTotal swaps: %d\n", swapCount);
```

```
Original array: 29 10 14 37 13
Swap 1: 10 29 14 37 13
Swap 2: 10 14 29 37 13
Swap 3: 10 14 29 13 37
Swap 4: 10 14 13 29 37
Swap 5: 10 13 14 29 37

Sorted array: 10 13 14 29 37
Total swaps: 5
```

OUTPUT

PRACTICE 2 : BUBBLE SORT

```
int main() {
    int arr[4] = {2, 4, 1, 3};
    int n = 4;
    int temp;
    int swapCount = 0;



    printf("Original array: ");
    for (int i = 0; i < n; i++) {
        printf("%d ", arr[i]);
    }
```

```c
for (int i = 0; i < n - 1; i++) {
        printf("\nPass %d:\n", i + 1);
        for (int j = 0; j < n - i - 1; j++) {
            if (arr[j] > arr[j + 1]) {
                //swap elements
                temp = arr[j];
                arr[j] = arr[j + 1];
                arr[j + 1] = temp;
                swapCount++;

                //print details
                            printf("  Swap %d: ", swapCount);
                for (int k = 0; k < n; k++) {
                    printf("%d ", arr[k]);
                }
                printf("\n");
            }
        }
    }
```

```c
printf("\nSorted array: ");
    for (int i = 0; i < n; i++) {
        printf("%d ", arr[i]);
    }
    printf("\nTotal swaps: %d\n", swapCount);
```

OUTPUT

```
Original array: 2 4 1 3

Pass 1:
  Swap 1: 2 1 4 3
  Swap 2: 2 1 3 4

Pass 2:
  Swap 3: 1 2 3 4

Pass 3:

Sorted array: 1 2 3 4
Total swaps: 3
```

EXERCISE 1: BUBBLE SORT

Sort the following value using Bubble Sort:

8 3 6 2 5

```
Original array: 8 3 6 2 5


Pass 1:
  Swap 1: 3 8 6 2 5
  Swap 2: 3 6 8 2 5
  Swap 3: 3 6 2 8 5
  Swap 4: 3 6 2 5 8

Pass 2:
  Swap 5: 3 2 6 5 8
  Swap 6: 3 2 5 6 8

Pass 3:
  Swap 7: 2 3 5 6 8

Pass 4:
Sorted!

Sorted array: 2 3 5 6 8
Total swaps: 7
```

EXERCISE 2: BUBBLE SORT

Sort the following value using Bubble Sort in Descending Order:

8 3 6 2 5

```
Original array: 8 3 6 2 5


Pass 1:
  Swap 1: 8 6 3 2 5
  Swap 2: 8 6 3 5 2

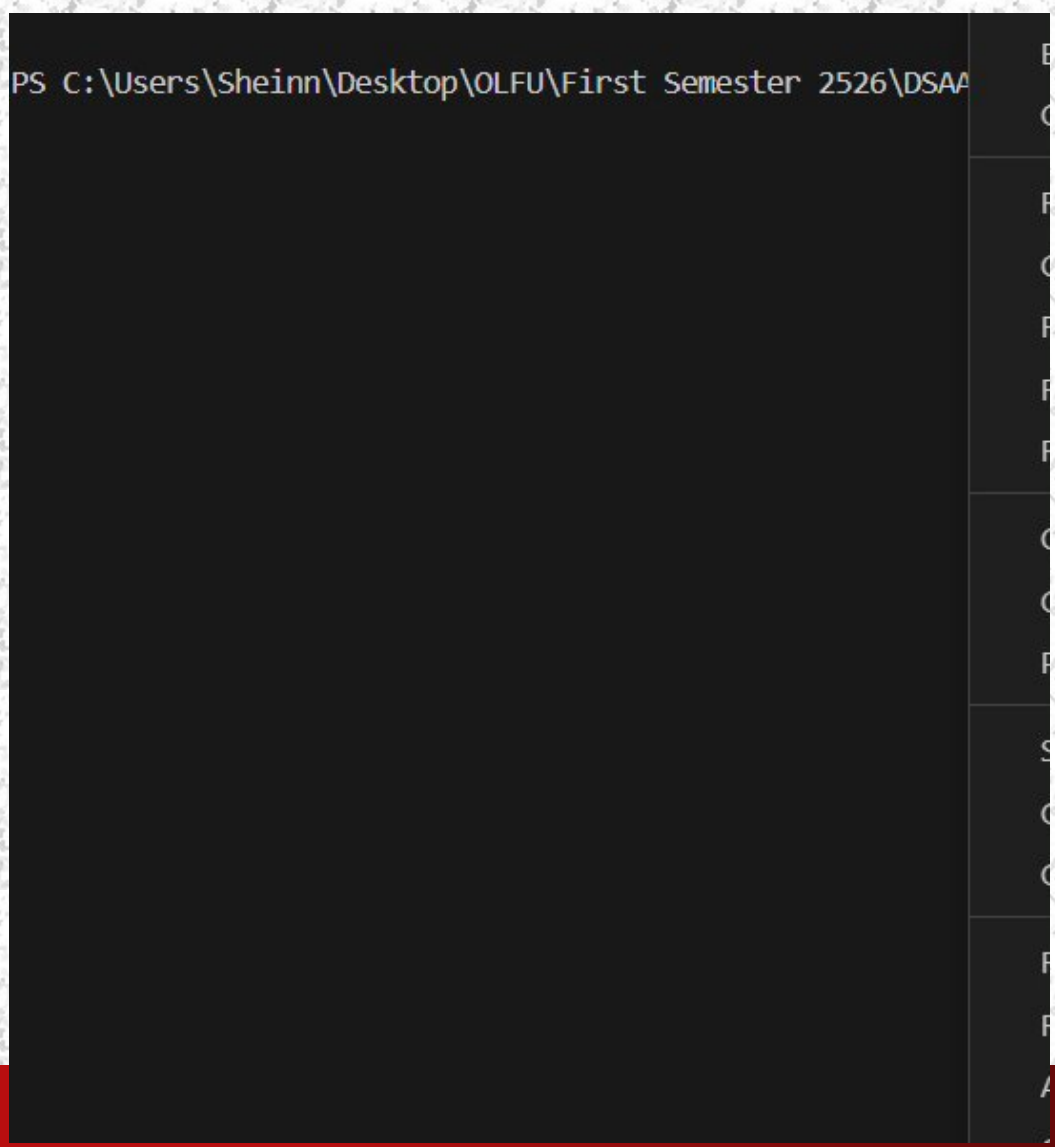
Pass 2:
  Swap 3: 8 6 5 3 2

Pass 3:
  No swaps in this pass, array is sorted early.

Sorted array (Descending): 8 6 5 3 2
Total swaps: 3
```

EXERCISE 3: BUBBLE SORT

Write, compile, and execute a C program that performs Bubble Sort on numbers entered by the user.



INSERTION SORTING : LABORATORY

DSAA 211: WEEK 8



PRACTICE 1 : INSERTION SORT

```
//initialization
int arr[5] = {29, 10, 14, 37, 13};
int n = 5;
int i, j, key;
//display original arrays
printf("Original array: ");
for (i = 0; i < n; i++) {
    printf("%d ", arr[i]);
}
printf("\n\n");
```

```
    for (i = 1; i < n; i++) {
        key = arr[i];
        j = i - 1;

        printf("Pass %d:\n", i);
        printf("  Key = %d\n", key);

//move elements greater than key to one position ahead
        while (j >= 0 && arr[j] > key) {
            arr[j + 1] = arr[j];
            j--;
        }
        arr[j + 1] = key;

        printf("  Result: ");
        for (int k = 0; k < n; k++) {
            printf("%d ", arr[k]);
        }
        printf("\n\n");
    }
```

```
printf("Sorted array (Ascending): ");
    for (i = 0; i < n; i++) {
        printf("%d ", arr[i]);
    }
    printf("\n");
```

```
Original array: 29 10 14 37 13

Pass 1:
  Key = 10
  Result: 10 29 14 37 13

Pass 2:
  Key = 14
  Result: 10 14 29 37 13

Pass 3:
  Key = 37
  Result: 10 14 29 37 13

Pass 4:
  Key = 13
  Result: 10 13 14 29 37

Sorted array (Ascending): 10 13 14 29 37
```

OUTPUT

PRACTICE 2 : INSERTION SORT

```
int arr[7] = {9, 5, 1, 4, 3, 8, 2};
int n = 7;
int i, j, key;

printf("Original array: ");
for (i = 0; i < n; i++) {
    printf("%d ", arr[i]);
}
printf("\n\n");
```

```c
for (i = 1; i < n; i++) {
        key = arr[i];
        j = i - 1;

        printf("Pass %d:\n", i);
        printf("  Key = %d\n", key);

        while (j >= 0 && arr[j] > key) {
            arr[j + 1] = arr[j];
            j--;
        }
        arr[j + 1] = key;

        printf("  Result: ");
        for (int k = 0; k < n; k++) {
            printf("%d ", arr[k]);
        }
        printf("\n\n");
    }
```

```c
    printf("Sorted array (Ascending): ");
    for (i = 0; i < n; i++) {
        printf("%d ", arr[i]);
    }
    printf("\n");
```

OUTPUT

```
Original array: 9 5 1 4 3 8 2

Pass 1:
  Key = 5
  Result: 5 9 1 4 3 8 2

Pass 2:
  Key = 1
  Result: 1 5 9 4 3 8 2

Pass 3:
  Key = 4
  Result: 1 4 5 9 3 8 2

Pass 4:
  Key = 3
  Result: 1 3 4 5 9 8 2

Pass 5:
  Key = 8
  Result: 1 3 4 5 8 9 2

Pass 6:
  Key = 2
  Result: 1 2 3 4 5 8 9

Sorted array (Ascending): 1 2 3 4 5 8 9
```

EXERCISE 1: INSERTION SORT

Sort the following value using Insertion Sort:

8 3 6 2 5

```
Original array: 8 3 6 2 5

Pass 1:
  Key = 3
  Result: 3 8 6 2 5

Pass 2:
  Key = 6
  Result: 3 6 8 2 5

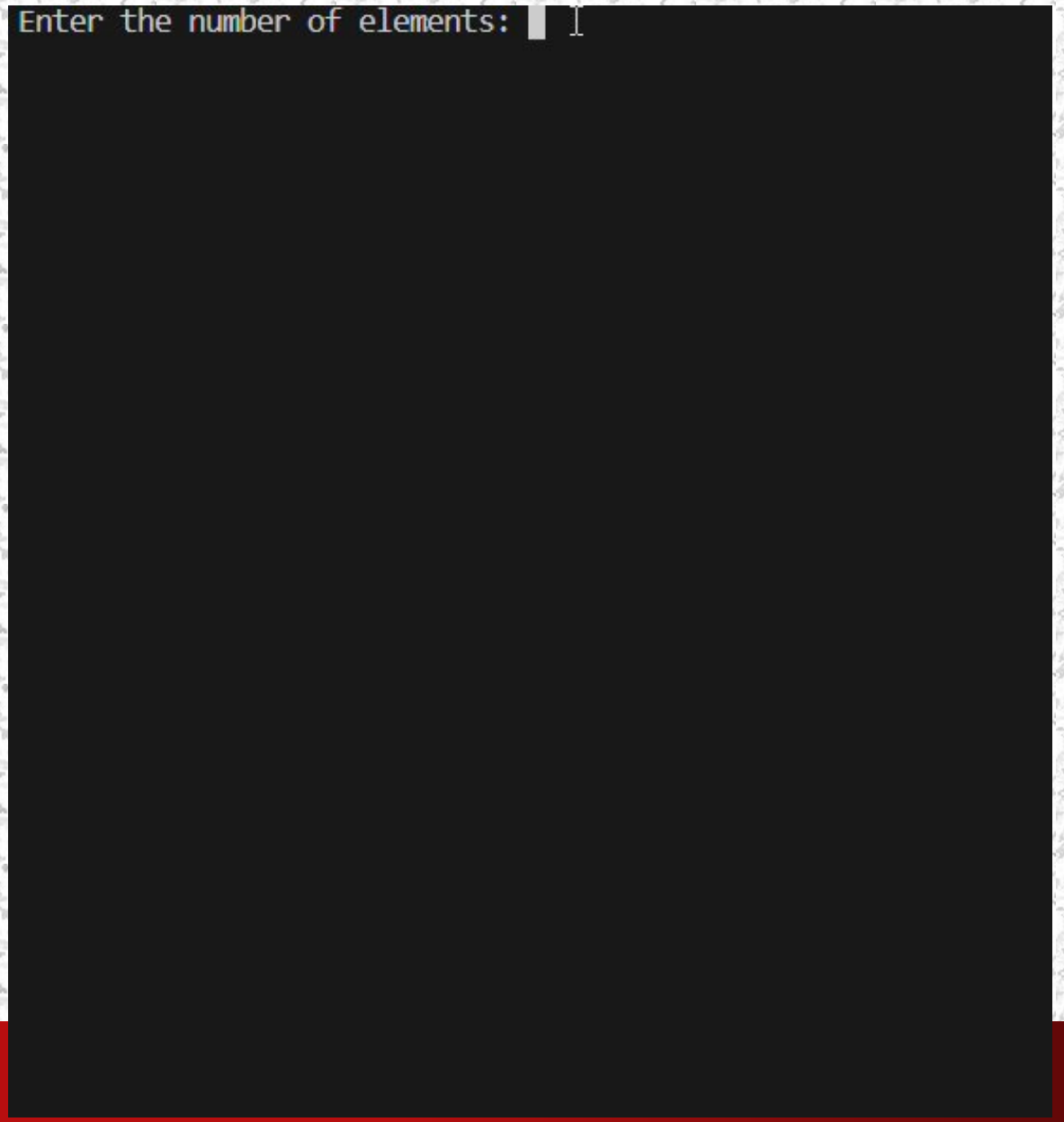
Pass 3:
  Key = 2
  Result: 2 3 6 8 5

Pass 4:
  Key = 5
  Result: 2 3 5 6 8

Sorted array (Ascending): 2 3 5 6 8
```

EXERCISE 2: INSERTION SORT

Write, compile, and execute a C program that performs Insertion Sort on numbers entered by the user.



THANK YOU!